

DOCUMENTATION COMPLÈTE DU PROJET DE SITE DE RECETTES DE CUISINE

Table des matières:

1. Vue d'ensemble du projet
2. Structure du projet
3. Dépendances principales
4. Fonctionnalités principales
5. Détail des composants
 - 5.1. Modèles (Models)
 - 5.2. Contrôleurs (Controllers)
 - 5.3. Routes
 - 5.4. Middlewares
6. Stockage des données
7. Fonctionnement technique
8. Guide de déploiement
9. Bonnes pratiques et optimisations

1. VUE D'ENSEMBLE DU PROJET

Ce projet est un site web de recettes de cuisine développé avec Node.js et Express. Il permet aux utilisateurs de consulter, créer, modifier et supprimer des recettes, ainsi que de gérer leurs recettes favorites. Le projet utilise une architecture MVC (Modèle-Vue-Contrôleur) et stocke les données dans des fichiers JSON plutôt que dans une base de données traditionnelle.

L'application est conçue pour être à la fois performante et facile à utiliser, avec une interface utilisateur intuitive et des fonctionnalités complètes de gestion de recettes. Elle implémente également un système d'authentification sécurisé pour protéger les données des utilisateurs.

2. STRUCTURE DU PROJET

=====

final/

- └─ app.js # Point d'entrée principal de l'application
- └─ index.js # Script de démarrage
- └─ package.json # Configuration du projet et dépendances
- └─ controllers/ # Contrôleurs pour la logique métier
 - | └─ recipeController.js
 - | └─ userController.js
- └─ data/ # Stockage des données JSON
 - | └─ recipes.json
 - | └─ users.json
- └─ middlewares/ # Middlewares Express
 - | └─ authMiddleware.js
- └─ models/ # Modèles de données
 - | └─ recipeModel.js
 - | └─ userModel.js
- └─ public/ # Fichiers statiques (CSS, JS, images)
 - | └─ uploads/ # Dossier pour les images téléchargées
- └─ routes/ # Définition des routes API
 - | └─ recipeRoutes.js
 - | └─ userRoutes.js
- └─ services/ # Services utilitaires
- └─ views/ # Vues (interface utilisateur)

=====

3. DÉPENDANCES PRINCIPALES

=====

Le projet utilise plusieurs bibliothèques Node.js pour implémenter ses fonctionnalités :

- express (^4.18.2) : Framework web pour Node.js qui simplifie la création d'applications web et d'APIs.

Utilisation : Gestion des routes, middlewares, et réponses HTTP.

- bcryptjs (^2.4.3) : Bibliothèque pour le hachage sécurisé des mots de passe.

Utilisation : Sécurisation des mots de passe utilisateur avant stockage.

- body-parser (^1.20.2) : Middleware pour analyser les corps de requêtes HTTP.

Utilisation : Extraction des données des requêtes POST, PUT, etc.

- cors (^2.8.5) : Middleware pour gérer le partage de ressources entre origines multiples.

Utilisation : Permettre l'accès à l'API depuis différentes origines.

- dotenv (^16.0.3) : Chargement des variables d'environnement depuis un fichier .env.

Utilisation : Configuration de l'application selon l'environnement.

- jsonwebtoken (^9.0.0) : Implémentation des JSON Web Tokens pour l'authentification.

Utilisation : Génération et vérification des tokens d'authentification.

- multer (^1.4.5-lts.1) : Middleware pour gérer les téléchargements de fichiers.

Utilisation : Traitement des images de recettes téléchargées.

- nodemon (^2.0.22, dev) : Utilitaire qui surveille les modifications de fichiers et redémarre automatiquement le serveur.

Utilisation : Faciliter le développement en rechargeant automatiquement l'application.

=====

4. FONCTIONNALITÉS PRINCIPALES

=====

4.1. Gestion des recettes

- Consultation de toutes les recettes avec filtrage, tri et pagination
- Recherche de recettes par mots-clés
- Affichage des recettes populaires
- Création de nouvelles recettes avec upload d'images
- Modification et suppression de recettes existantes
- Incrémentation du compteur de vues des recettes
- Commentaires sur les recettes

4.2. Gestion des utilisateurs

- Inscription et connexion des utilisateurs
- Gestion de profil utilisateur
- Changement de mot de passe
- Authentification via JWT (JSON Web Tokens)

4.3. Gestion des favoris

- Ajout de recettes aux favoris
- Suppression de recettes des favoris
- Consultation de la liste de recettes favorites
- Vérification si une recette est dans les favoris

4.4. Interface utilisateur

- Interface responsive adaptée aux mobiles et ordinateurs
- Formulaire de création et modification de recettes
- Affichage détaillé des recettes
- Système de navigation et de recherche
- Gestion du profil utilisateur

=====

5. DÉTAIL DES COMPOSANTS

=====

5.1. MODÈLES (MODELS)

5.1.1. RecipeModel.js

Ce modèle gère toutes les opérations liées aux recettes dans l'application.

Principales fonctions :

* readRecipesFile() :

- Description : Lit le fichier de recettes JSON et retourne son contenu.
- Paramètres : Aucun
- Retour : Promise<Array> - Liste des recettes
- Fonctionnement : Utilise fs.promises.readFile pour lire le fichier de manière asynchrone, puis parse le JSON.
- Gestion d'erreur : En cas d'erreur, retourne un tableau vide et log l'erreur.

* getAllRecipes(filters, sort, skip, limit) :

- Description : Récupère toutes les recettes avec filtrage, tri et pagination.
- Paramètres :
 - * filters (Object) : Critères de filtrage (search, maxPrepTime)
 - * sort (String) : Champ de tri (views, title, createdAt)
 - * skip (Number) : Nombre d'éléments à sauter (pagination)
 - * limit (Number) : Nombre d'éléments à retourner (pagination)
- Retour : Promise<Array> - Liste des recettes filtrées et triées
- Fonctionnement :
 1. Lit le fichier de recettes

2. Applique les filtres (recherche textuelle, temps de préparation max)
3. Trie les résultats selon le critère spécifié
4. Applique la pagination (skip + limit)

- Exemple d'utilisation :

```
```js
const recipes = await RecipeModel.getAllRecipes({search: 'poulet'}, 'views', 0, 10);
```
```

* countRecipes(filters) :

- Description : Compte le nombre total de recettes correspondant aux filtres.
- Paramètres : filters (Object) - Critères de filtrage
- Retour : Promise<Number> - Nombre de recettes
- Fonctionnement : Similaire à getAllRecipes mais retourne uniquement le nombre d'éléments.

* getRecipeById(id) :

- Description : Récupère une recette par son ID.
- Paramètres : id (String) - ID de la recette
- Retour : Promise<Object|null> - Recette trouvée ou null
- Fonctionnement : Filtre toutes les recettes pour trouver celle avec l'ID correspondant.

* createRecipe(recipeData) :

- Description : Crée une nouvelle recette.
- Paramètres : recipeData (Object) - Données de la recette
- Retour : Promise<Object> - Recette créée
- Fonctionnement :
 1. Lit toutes les recettes existantes
 2. Crée un nouvel objet recette avec un ID unique (timestamp)
 3. Ajoute la recette au tableau
 4. Écrit le tableau mis à jour dans le fichier JSON

- Exemple d'utilisation :

```
```js
```

```
const newRecipe = await RecipeModel.createRecipe({
 title: 'Poulet rôti',
 description: 'Délicieux poulet rôti au four',
 ingredients: ['1 poulet', 'Herbes', 'Sel', 'Poivre'],
 instructions: ['Préchauffer le four', 'Assaisonner le poulet', 'Cuire 1h30']
});
...

```

\* updateRecipe(id, recipeData) :

- Description : Met à jour une recette existante.
- Paramètres :
  - \* id (String) - ID de la recette
  - \* recipeData (Object) - Nouvelles données
- Retour : Promise<Object|null> - Recette mise à jour ou null
- Fonctionnement :
  1. Trouve la recette par son ID
  2. Fusionne les données existantes avec les nouvelles
  3. Met à jour le fichier JSON

\* deleteRecipe(id) :

- Description : Supprime une recette.
- Paramètres : id (String) - ID de la recette
- Retour : Promise<Boolean> - True si supprimée, false sinon
- Fonctionnement : Filtre les recettes pour exclure celle avec l'ID spécifié, puis écrit le résultat.

\* incrementViews(id) :

- Description : Incrémente le compteur de vues d'une recette.
- Paramètres : id (String) - ID de la recette
- Retour : Promise<Boolean> - True si mise à jour réussie
- Fonctionnement : Trouve la recette, incrémente son compteur de vues, puis sauvegarde.

\* addComment(recipeId, commentData) :

- Description : Ajoute un commentaire à une recette.

- Paramètres :

\* recipeId (String) - ID de la recette

\* commentData (Object) - Données du commentaire (texte, auteur, etc.)

- Retour : Promise<Object> - Commentaire ajouté

- Fonctionnement : Trouve la recette, ajoute le commentaire à son tableau de commentaires, puis sauvegarde.

\* getRecipesByUser(userId) :

- Description : Récupère toutes les recettes créées par un utilisateur spécifique.

- Paramètres : userId (String) - ID de l'utilisateur

- Retour : Promise<Array> - Liste des recettes de l'utilisateur

- Fonctionnement : Filtre toutes les recettes pour trouver celles créées par l'utilisateur spécifié.

### 5.1.2. UserModel.js

-----

Ce modèle gère toutes les opérations liées aux utilisateurs dans l'application.

Principales fonctions :

\* getAllUsers() :

- Description : Récupère tous les utilisateurs.

- Paramètres : Aucun

- Retour : Promise<Array> - Liste des utilisateurs

- Fonctionnement : Lit le fichier JSON des utilisateurs.

\* getUserById(id) :

- Description : Récupère un utilisateur par son ID.

- Paramètres : id (String) - ID de l'utilisateur

- Retour : Promise<Object|null> - Utilisateur trouvé ou null

- Fonctionnement : Filtre les utilisateurs pour trouver celui avec l'ID correspondant.

\* getUserByEmail(email) :

- Description : Récupère un utilisateur par son email.
- Paramètres : email (String) - Email de l'utilisateur
- Retour : Promise<Object|null> - Utilisateur trouvé ou null
- Fonctionnement : Filtre les utilisateurs pour trouver celui avec l'email correspondant.

\* createUser(userData) :

- Description : Crée un nouvel utilisateur.
- Paramètres : userData (Object) - Données de l'utilisateur
- Retour : Promise<Object> - Utilisateur créé (sans le mot de passe)
- Fonctionnement :
  1. Vérifie si l'email existe déjà
  2. Hache le mot de passe avec bcrypt
  3. Crée un nouvel utilisateur avec un ID unique
  4. Sauvegarde dans le fichier JSON

- Exemple d'utilisation :

```
```js
const newUser = await UserModel.createUser({
  name: 'John Doe',
  email: 'john@example.com',
  password: 'motdepasse123'
});
```
```

\* updateUser(id, userData) :

- Description : Met à jour un utilisateur existant.
- Paramètres :
  - \* id (String) - ID de l'utilisateur
  - \* userData (Object) - Nouvelles données

- Retour : Promise<Object|null> - Utilisateur mis à jour (sans le mot de passe) ou null

- Fonctionnement :

1. Trouve l'utilisateur par son ID
2. Si un nouveau mot de passe est fourni, le hache
3. Fusionne les données existantes avec les nouvelles
4. Sauvegarde dans le fichier JSON

\* deleteUser(id) :

- Description : Supprime un utilisateur.

- Paramètres : id (String) - ID de l'utilisateur

- Retour : Promise<Boolean> - True si supprimé, false sinon

- Fonctionnement : Filtre les utilisateurs pour exclure celui avec l'ID spécifié, puis écrit le résultat.

\* verifyCredentials(email, password) :

- Description : Vérifie les identifiants de connexion.

- Paramètres :

\* email (String) - Email de l'utilisateur

\* password (String) - Mot de passe à vérifier

- Retour : Promise<Object|null> - Utilisateur authentifié (sans le mot de passe) ou null

- Fonctionnement :

1. Récupère l'utilisateur par son email
2. Compare le mot de passe fourni avec le hash stocké (via bcrypt.compare)
3. Retourne l'utilisateur sans son mot de passe si authentifié

- Exemple d'utilisation :

```
```js
```

```
const user = await UserModel.verifyCredentials('john@example.com', 'motdepasse123');
```

```
if (user){
```

```
  // Authentification réussie
```

```
}
```

```
```
```

\* getUserFavorites(userId) :

- Description : Récupère les recettes favorites d'un utilisateur.
- Paramètres : userId (String) - ID de l'utilisateur
- Retour : Promise<Array> - Liste des recettes favorites
- Fonctionnement :
  1. Récupère l'utilisateur par son ID
  2. Pour chaque ID de recette favorite, récupère les détails complets de la recette
  3. Retourne la liste des recettes favorites complètes

\* addFavorite(userId, recipeId) :

- Description : Ajoute une recette aux favoris d'un utilisateur.
- Paramètres :
  - \* userId (String) - ID de l'utilisateur
  - \* recipeId (String) - ID de la recette
- Retour : Promise<Boolean> - True si ajouté, false sinon
- Fonctionnement : Met à jour le tableau des favoris de l'utilisateur et sauvegarde.

\* removeFavorite(userId, recipeId) :

- Description : Supprime une recette des favoris d'un utilisateur.
- Paramètres :
  - \* userId (String) - ID de l'utilisateur
  - \* recipeId (String) - ID de la recette
- Retour : Promise<Boolean> - True si supprimé, false sinon
- Fonctionnement : Filtre le tableau des favoris pour exclure la recette spécifiée et sauvegarde.

=====

## 5.2. CONTRÔLEURS (CONTROLLERS)

-----

### 5.2.1. recipeController.js

-----

Ce contrôleur gère toutes les opérations liées aux recettes dans l'API.

Principales fonctions :

\* getAllRecipes(req, res) :

- Description : Récupère toutes les recettes avec filtres, pagination et tri.

- Paramètres :

\* req (Object) - Requête Express avec les paramètres de requête (query)

\* res (Object) - Réponse Express

- Fonctionnement :

1. Extrait les paramètres de pagination (page, limit)

2. Extrait les paramètres de filtrage (search, maxPrepTime)

3. Extrait le paramètre de tri (sort)

4. Appelle RecipeModel.getAllRecipes avec ces paramètres

5. Si l'utilisateur est connecté, marque les recettes favorites

6. Renvoie une réponse JSON avec les recettes et les métadonnées de pagination

- Exemple de requête : GET /api/recipes?page=1&limit=10&search=poulet&sort=views

- Exemple de réponse :

```
```json
{
  "success": true,
  "total": 25,
  "page": 1,
  "limit": 10,
  "totalPages": 3,
  "recipes": [...]
}
```
```

\* getRecipeById(req, res) :

- Description : Récupère une recette par son ID.

- Paramètres :

\* req (Object) - Requête Express avec l'ID de la recette dans les paramètres

\* res (Object) - Réponse Express

- Fonctionnement :

1. Extrait l'ID de la recette des paramètres de route

2. Appelle RecipeModel.getRecipeById avec cet ID

3. Si la recette n'existe pas, renvoie une erreur 404

4. Si l'utilisateur est connecté, vérifie si la recette est dans ses favoris

5. Incrémente le compteur de vues de la recette

6. Renvoie la recette en réponse

- Exemple de requête : GET /api/recipes/123456789

- Gestion d'erreur : Renvoie une erreur 404 si la recette n'existe pas, 500 en cas d'erreur serveur.

\* createRecipe(req, res) :

- Description : Crée une nouvelle recette.

- Paramètres :

\* req (Object) - Requête Express avec les données de la recette dans le corps

\* res (Object) - Réponse Express

- Fonctionnement :

1. Vérifie que l'utilisateur est authentifié

2. Valide les données de la recette (titre obligatoire)

3. Prépare les données de la recette (conversion des chaînes en tableaux pour ingrédients/instructions)

4. Ajoute les informations de l'utilisateur (ID, nom)

5. Gère l'image téléchargée si présente

6. Appelle RecipeModel.createRecipe avec les données préparées

7. Renvoie la nouvelle recette créée

- Exemple de requête : POST /api/recipes avec données multipart/form-data

- Validation : Vérifie que le titre est présent, authentification requise.

\* updateRecipe(req, res) :

- Description : Met à jour une recette existante.
- Paramètres :
  - \* req (Object) - Requête Express avec l'ID de la recette et les nouvelles données
  - \* res (Object) - Réponse Express
- Fonctionnement :
  1. Vérifie que la recette existe
  2. Vérifie que l'utilisateur est authentifié et est le créateur de la recette
  3. Prépare les données à mettre à jour (conversion des types)
  4. Gère l'image téléchargée si présente
  5. Appelle RecipeModel.updateRecipe avec l'ID et les données
  6. Renvoie la recette mise à jour
- Exemple de requête : PUT /api/recipes/123456789 avec données multipart/form-data
- Sécurité : Vérifie que l'utilisateur est le créateur de la recette.

\* deleteRecipe(req, res) :

- Description : Supprime une recette.
- Paramètres :
  - \* req (Object) - Requête Express avec l'ID de la recette
  - \* res (Object) - Réponse Express
- Fonctionnement :
  1. Vérifie que la recette existe
  2. Vérifie que l'utilisateur est authentifié et est le créateur de la recette
  3. Appelle RecipeModel.deleteRecipe avec l'ID
  4. Renvoie un message de succès
- Exemple de requête : DELETE /api/recipes/123456789
- Sécurité : Vérifie que l'utilisateur est le créateur de la recette.

\* searchRecipes(req, res) :

- Description : Recherche des recettes par mot-clé.
- Paramètres :
  - \* req (Object) - Requête Express avec le terme de recherche

\* res (Object) - Réponse Express

- Fonctionnement :

1. Extrait le terme de recherche de la requête
2. Appelle RecipeModel.getAllRecipes avec le filtre de recherche
3. Renvoie les recettes correspondantes

- Exemple de requête : GET /api/recipes/search?q=poulet

\* getPopularRecipes(req, res) :

- Description : Récupère les recettes populaires (les plus vues).

- Paramètres :

\* req (Object) - Requête Express

\* res (Object) - Réponse Express

- Fonctionnement :

1. Appelle RecipeModel.getAllRecipes avec un tri par vues
2. Limite le nombre de résultats (généralement 5 ou 10)
3. Renvoie les recettes les plus populaires

- Exemple de requête : GET /api/recipes/popular

#### 5.2.2. userController.js (dédit)

-----

Ce contrôleur gère toutes les opérations liées aux utilisateurs dans l'API.

Principales fonctions :

\* register(req, res) :

- Description : Inscrit un nouvel utilisateur.

- Paramètres :

\* req (Object) - Requête Express avec les données d'inscription

\* res (Object) - Réponse Express

- Fonctionnement :

1. Valide les données d'inscription (email, mot de passe, nom)

2. Vérifie que l'email n'est pas déjà utilisé
3. Appelle `UserModel.createUser` avec les données validées
4. Génère un token JWT pour l'authentification
5. Renvoie l'utilisateur créé et le token

- Exemple de requête : `POST /api/users/register` avec données JSON

- Validation : Vérifie que l'email est valide, que le mot de passe est suffisamment fort.

\* `login(req, res)` :

- Description : Connecte un utilisateur existant.

- Paramètres :

\* `req (Object)` - Requête Express avec les identifiants

\* `res (Object)` - Réponse Express

- Fonctionnement :

1. Extrait l'email et le mot de passe de la requête
2. Appelle `UserModel.verifyCredentials` pour vérifier les identifiants
3. Si les identifiants sont valides, génère un token JWT
4. Renvoie l'utilisateur et le token

- Exemple de requête : `POST /api/users/login` avec données JSON

- Sécurité : Utilise `bcrypt` pour comparer les mots de passe de manière sécurisée.

\* `getProfile(req, res)` :

- Description : Récupère le profil de l'utilisateur connecté.

- Paramètres :

\* `req (Object)` - Requête Express avec les informations de l'utilisateur (ajoutées par le middleware d'authentification)

\* `res (Object)` - Réponse Express

- Fonctionnement :

1. Récupère l'ID utilisateur depuis `req.user`
2. Appelle `UserModel.getUserById` avec cet ID
3. Renvoie les informations de profil

- Exemple de requête : `GET /api/users/profile` (avec token JWT)

- Sécurité : Nécessite une authentification via le middleware.

\* updateProfile(req, res) :

- Description : Met à jour le profil de l'utilisateur connecté.

- Paramètres :

- \* req (Object) - Requête Express avec les nouvelles données de profil

- \* res (Object) - Réponse Express

- Fonctionnement :

- 1. Récupère l'ID utilisateur depuis req.user

- 2. Valide les données de mise à jour

- 3. Appelle UserModel.updateUser avec l'ID et les données

- 4. Renvoie le profil mis à jour

- Exemple de requête : PUT /api/users/profile (avec token JWT)

- Sécurité : Nécessite une authentification via le middleware.

\* changePassword(req, res) :

- Description : Change le mot de passe de l'utilisateur connecté.

- Paramètres :

- \* req (Object) - Requête Express avec l'ancien et le nouveau mot de passe

- \* res (Object) - Réponse Express

- Fonctionnement :

- 1. Récupère l'ID utilisateur depuis req.user

- 2. Vérifie que l'ancien mot de passe est correct

- 3. Valide le nouveau mot de passe

- 4. Appelle UserModel.updateUser pour mettre à jour le mot de passe

- 5. Renvoie un message de succès

- Exemple de requête : PUT /api/users/change-password (avec token JWT)

- Sécurité : Vérifie l'ancien mot de passe avant de permettre le changement.

\* getFavorites(req, res) :

- Description : Récupère les recettes favorites de l'utilisateur connecté.

- Paramètres :

\* req (Object) - Requête Express

\* res (Object) - Réponse Express

- Fonctionnement :

1. Récupère l'ID utilisateur depuis req.user
2. Appelle UserModel.getUserFavorites avec cet ID
3. Renvoie la liste des recettes favorites

- Exemple de requête : GET /api/users/favorites (avec token JWT)

\* addFavorite(req, res) :

- Description : Ajoute une recette aux favoris de l'utilisateur connecté.

- Paramètres :

\* req (Object) - Requête Express avec l'ID de la recette

\* res (Object) - Réponse Express

- Fonctionnement :

1. Récupère l'ID utilisateur depuis req.user
2. Récupère l'ID de la recette depuis les paramètres de route
3. Vérifie que la recette existe
4. Appelle UserModel.addFavorite avec les deux IDs
5. Renvoie un message de succès

- Exemple de requête : POST /api/users/favorites/123456789 (avec token JWT)

\* removeFavorite(req, res) :

- Description : Supprime une recette des favoris de l'utilisateur connecté.

- Paramètres :

\* req (Object) - Requête Express avec l'ID de la recette

\* res (Object) - Réponse Express

- Fonctionnement :

1. Récupère l'ID utilisateur depuis req.user
2. Récupère l'ID de la recette depuis les paramètres de route
3. Appelle UserModel.removeFavorite avec les deux IDs

4. Renvoie un message de succès

- Exemple de requête : DELETE /api/users/favorites/123456789 (avec token JWT)

\* checkFavorite(req, res) :

- Description : Vérifie si une recette est dans les favoris de l'utilisateur.

- Paramètres :

\* req (Object) - Requête Express avec l'ID de la recette

\* res (Object) - Réponse Express

- Fonctionnement :

1. Récupère l'ID utilisateur depuis req.user

2. Récupère l'ID de la recette depuis les paramètres de route

3. Récupère les favoris de l'utilisateur

4. Vérifie si la recette est dans les favoris

5. Renvoie un booléen indiquant si la recette est favorite

- Exemple de requête : GET /api/users/favorites/check/123456789 (avec token JWT)

=====

### 5.3. ROUTES

-----

#### 5.3.1. recipeRoutes.js

-----

Ce fichier définit toutes les routes liées aux recettes dans l'API.

Configuration et middlewares :

- Utilise express.Router() pour créer un routeur

- Configure multer pour gérer les téléchargements d'images :

\* Stockage des images dans le dossier 'public/uploads'

\* Nommage des fichiers avec timestamp pour éviter les conflits

\* Filtre pour n'accepter que les images

\* Limite de taille à 5 Mo

## Routes définies :

### \* GET / :

- Fonction : `recipeController.getAllRecipes`
- Description : Récupère toutes les recettes avec filtrage, tri et pagination
- Accès : Public
- Paramètres de requête :
  - \* `page` : Numéro de page (défaut: 1)
  - \* `limit` : Nombre d'éléments par page (défaut: 10)
  - \* `search` : Terme de recherche
  - \* `maxPrepTime` : Temps de préparation maximum
  - \* `sort` : Champ de tri (views, title, createdAt)

### \* GET /search :

- Fonction : `recipeController.searchRecipes`
- Description : Recherche des recettes par mot-clé
- Accès : Public
- Paramètres de requête :
  - \* `q` : Terme de recherche

### \* GET /popular :

- Fonction : `recipeController.getPopularRecipes`
- Description : Récupère les recettes les plus populaires
- Accès : Public

### \* GET /:id :

- Fonction : `recipeController.getRecipeById`
- Description : Récupère une recette par son ID
- Accès : Public
- Paramètres de route :
  - \* `id` : ID de la recette

\* POST / :

- Fonction : `recipeController.createRecipe`
- Description : Crée une nouvelle recette
- Accès : Protégé (authentification requise)
- Middlewares :
  - \* `authMiddleware` : Vérifie l'authentification
  - \* `upload.single('image')` : Gère le téléchargement d'image
- Corps de requête :
  - \* `title` : Titre de la recette (obligatoire)
  - \* `description` : Description de la recette
  - \* `ingredients` : Ingrédients (chaîne ou tableau)
  - \* `instructions` : Instructions (chaîne ou tableau)
  - \* `prepTime` : Temps de préparation en minutes
  - \* `cookTime` : Temps de cuisson en minutes
  - \* `servings` : Nombre de portions
  - \* `image` : Fichier image

\* PUT /:id :

- Fonction : `recipeController.updateRecipe`
- Description : Met à jour une recette existante
- Accès : Protégé (authentification requise)
- Middlewares :
  - \* `authMiddleware` : Vérifie l'authentification
  - \* `upload.single('image')` : Gère le téléchargement d'image
- Paramètres de route :
  - \* `id` : ID de la recette
- Corps de requête : Mêmes champs que pour la création

\* DELETE /:id :

- Fonction : `recipeController.deleteRecipe`

- Description : Supprime une recette
- Accès : Protégé (authentification requise)
- Middleware : authMiddleware
- Paramètres de route :
  - \* id : ID de la recette

### 5.3.2. userRoutes.js

-----

Ce fichier définit toutes les routes liées aux utilisateurs dans l'API.

Routes définies :

\* POST /register :

- Fonction : userController.register
- Description : Inscrit un nouvel utilisateur
- Accès : Public
- Corps de requête :
  - \* name : Nom de l'utilisateur
  - \* email : Email de l'utilisateur
  - \* password : Mot de passe

\* POST /login :

- Fonction : userController.login
- Description : Connecte un utilisateur existant
- Accès : Public
- Corps de requête :
  - \* email : Email de l'utilisateur
  - \* password : Mot de passe

\* GET /profile :

- Fonction : userController.getProfile

- Description : Récupère le profil de l'utilisateur connecté
- Accès : Protégé (authentification requise)
- Middleware : authMiddleware

\* PUT /profile :

- Fonction : userController.updateProfile
- Description : Met à jour le profil de l'utilisateur connecté
- Accès : Protégé (authentification requise)
- Middleware : authMiddleware
- Corps de requête :
  - \* name : Nouveau nom
  - \* email : Nouvel email

\* PUT /change-password :

- Fonction : userController.changePassword
- Description : Change le mot de passe de l'utilisateur connecté
- Accès : Protégé (authentification requise)
- Middleware : authMiddleware
- Corps de requête :
  - \* currentPassword : Mot de passe actuel
  - \* newPassword : Nouveau mot de passe

\* GET /favorites :

- Fonction : userController.getFavorites
- Description : Récupère les recettes favorites de l'utilisateur connecté
- Accès : Protégé (authentification requise)
- Middleware : authMiddleware

\* GET /favorites/check/:recipeId :

- Fonction : userController.checkFavorite
- Description : Vérifie si une recette est dans les favoris de l'utilisateur

- Accès : Protégé (authentification requise)

- Middleware : authMiddleware

- Paramètres de route :

  - \* recipeld : ID de la recette

\* POST /favorites/:recipeld :

- Fonction : userController.addFavorite

- Description : Ajoute une recette aux favoris de l'utilisateur

- Accès : Protégé (authentification requise)

- Middleware : authMiddleware

- Paramètres de route :

  - \* recipeld : ID de la recette

\* DELETE /favorites/:recipeld :

- Fonction : userController.removeFavorite

- Description : Supprime une recette des favoris de l'utilisateur

- Accès : Protégé (authentification requise)

- Middleware : authMiddleware

- Paramètres de route :

  - \* recipeld : ID de la recette

=====

## 5.4. MIDDLEWARES

-----

### 5.4.1. authMiddleware.js (déduit)

-----

Ce middleware gère l'authentification des utilisateurs pour les routes protégées.

Fonctionnement :

#### 1. Extraction du token JWT :

- Récupère le token depuis l'en-tête Authorization (format "Bearer [token]")
- Si aucun token n'est fourni, renvoie une erreur 401 (Non autorisé)

#### 2. Vérification du token :

- Utilise `jsonwebtoken.verify` pour valider le token avec la clé secrète
- Si le token est invalide ou expiré, renvoie une erreur 401

#### 3. Récupération des informations utilisateur :

- Extrait l'ID utilisateur du payload du token
- Récupère les informations complètes de l'utilisateur depuis la base de données
- Si l'utilisateur n'existe pas, renvoie une erreur 401

#### 4. Ajout des informations utilisateur à la requête :

- Ajoute l'objet utilisateur à `req.user` pour utilisation dans les contrôleurs
- Appelle `next()` pour passer au middleware ou contrôleur suivant

Exemple d'utilisation dans les routes :

```
`` `js
router.get('/profile', authMiddleware, userController.getProfile);
`` `
```

Exemple de code (reconstitué) :

```
`` `js
const jwt = require('jsonwebtoken');
const UserModel = require('../models/userModel');

module.exports = async (req, res, next) => {
 try {
 // Extraction du token
 const authHeader = req.headers.authorization;
```

```
if (!authHeader || !authHeader.startsWith('Bearer ')) {
 return res.status(401).json({
 success: false,
 message: 'Accès non autorisé, token manquant'
 });
}
```

```
const token = authHeader.split(' ')[1];
```

```
// Vérification du token
```

```
const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret_key');
```

```
// Récupération de l'utilisateur
```

```
const user = await UserModel.getUserById(decoded.id);
```

```
if (!user) {
```

```
 return res.status(401).json({
```

```
 success: false,
```

```
 message: 'Utilisateur non trouvé'
```

```
 });
```

```
}
```

```
// Ajout de l'utilisateur à la requête
```

```
req.user = user;
```

```
next();
```

```
} catch (error) {
```

```
 console.error('Erreur d\'authentification:', error);
```

```
 res.status(401).json({
```

```
 success: false,
```

```
 message: 'Token invalide ou expiré',
```

```
 error: error.message
```

```
 });
```

```
}
};
...
```

## =====

## 6. STOCKAGE DES DONNÉES

-----

Le projet utilise des fichiers JSON pour stocker les données plutôt qu'une base de données traditionnelle. Cette approche simplifie le déploiement mais présente des limitations en termes de performances et de concurrence.

### 6.1. Structure des fichiers de données

-----

\* recipes.json :

- Stocke un tableau d'objets représentant les recettes

- Chaque recette contient :

- \* id : Identifiant unique (timestamp)

- \* title : Titre de la recette

- \* description : Description de la recette

- \* ingredients : Tableau d'ingrédients

- \* instructions : Tableau d'instructions

- \* prepTime : Temps de préparation en minutes

- \* cookTime : Temps de cuisson en minutes

- \* servings : Nombre de portions

- \* userId : ID du créateur

- \* author : Nom du créateur

- \* image : Chemin vers l'image

- \* views : Nombre de vues

- \* comments : Tableau de commentaires

- \* createdAt : Date de création

- \* updatedAt : Date de dernière mise à jour

- \* users.json :

- Stocke un tableau d'objets représentant les utilisateurs

- Chaque utilisateur contient :

- \* id : Identifiant unique (timestamp)

- \* name : Nom de l'utilisateur

- \* email : Email de l'utilisateur (unique)

- \* password : Mot de passe haché (avec bcrypt)

- \* favorites : Tableau d'IDs de recettes favorites

- \* createdAt : Date de création

- \* updatedAt : Date de dernière mise à jour

## 6.2. Gestion des fichiers

-----

Le projet utilise le module fs (file system) de Node.js pour manipuler les fichiers JSON :

- \* Lecture des données :

- Utilise fs.promises.readFile pour lire les fichiers de manière asynchrone

- Parse le contenu JSON avec JSON.parse()

- \* Écriture des données :

- Convertit les données en JSON avec JSON.stringify()

- Utilise fs.promises.writeFile pour écrire dans les fichiers de manière asynchrone

- Utilise un formatage avec indentation (null, 2) pour la lisibilité

- \* Création initiale des fichiers :

- Vérifie si les fichiers existent au démarrage

- Crée les fichiers avec un tableau vide s'ils n'existent pas

### 6.3. Limitations et considérations

-----

#### \* Concurrence :

- Risque de conflits lors d'écritures simultanées
- Pas de verrouillage de fichiers ou de transactions

#### \* Performance :

- Chargement complet du fichier pour chaque opération
- Peut devenir lent avec un grand nombre d'enregistrements

#### \* Sécurité :

- Les mots de passe sont correctement hachés avec bcrypt
- Les données sensibles sont exclues des réponses API

#### \* Évolutivité :

- Solution adaptée pour de petites applications
- Pour une application en production avec beaucoup d'utilisateurs, une base de données comme MongoDB ou PostgreSQL serait préférable

=====

## 7. FONCTIONNEMENT TECHNIQUE

-----

### 7.1. Flux d'exécution

-----

Le flux d'exécution typique d'une requête dans l'application est le suivant :

1. Le client envoie une requête HTTP à l'API
2. Le serveur Express reçoit la requête
3. Les middlewares globaux sont exécutés (cors, body-parser, etc.)

4. Le routeur dirige la requête vers la route appropriée
5. Si la route est protégée, le middleware d'authentification vérifie le token JWT
6. Le contrôleur correspondant traite la requête
7. Le contrôleur utilise le modèle pour accéder aux données
8. Le modèle lit/écrit dans les fichiers JSON
9. Le contrôleur prépare et envoie la réponse au client

## 7.2. Gestion de l'authentification

-----

L'authentification est basée sur les JSON Web Tokens (JWT) :

### 1. Inscription/Connexion :

- L'utilisateur s'inscrit ou se connecte
- Le serveur vérifie les identifiants
- Si valides, un token JWT est généré avec l'ID utilisateur
- Le token est renvoyé au client

### 2. Accès aux routes protégées :

- Le client inclut le token dans l'en-tête Authorization
- Le middleware d'authentification vérifie le token
- Si valide, les informations utilisateur sont ajoutées à la requête
- Le contrôleur peut alors accéder à req.user

### 3. Sécurité :

- Les tokens ont une durée de validité limitée
- Les mots de passe sont hachés avec bcrypt
- Les informations sensibles sont exclues des réponses

## 7.3. Gestion des fichiers

-----

Le téléchargement et la gestion des images de recettes sont gérés par multer :

#### 1. Configuration :

- Stockage des images dans le dossier public/uploads
- Nommage des fichiers avec timestamp pour éviter les conflits
- Filtre pour n'accepter que les images
- Limite de taille à 5 Mo

#### 2. Processus de téléchargement :

- Le client envoie un formulaire multipart/form-data
- Multer intercepte le fichier image
- L'image est enregistrée sur le disque
- Le chemin de l'image est ajouté à la recette

#### 3. Accès aux images :

- Les images sont servies comme fichiers statiques
- Le chemin relatif est stocké dans la base de données
- Le client peut accéder aux images via leur URL

#### 7.4. Gestion des erreurs

-----

L'application implémente une gestion d'erreurs robuste :

#### 1. Try/catch dans les contrôleurs :

- Toutes les fonctions asynchrones sont entourées de try/catch
- Les erreurs sont loguées et une réponse appropriée est envoyée

#### 2. Middleware de gestion d'erreurs global :

- Capture les erreurs non gérées

- Envoie une réponse 500 avec un message d'erreur

### 3. Gestion des routes non trouvées :

- Un middleware 404 capture les requêtes vers des routes inexistantes
- Renvoie une réponse 404 avec un message approprié

=====

## 8. GUIDE DE DÉPLOIEMENT

-----

### 8.1. Prérequis

-----

- Node.js (v14.x ou supérieur)
- npm (v6.x ou supérieur)
- Espace disque pour les images téléchargées

### 8.2. Installation locale

-----

#### 1. Cloner le dépôt :

```

git clone [URL_DU_DEPOT]

cd final

```

#### 2. Installer les dépendances :

```

npm install

```

### 3. Configurer les variables d'environnement :

- Créer un fichier .env à la racine du projet
- Ajouter les variables suivantes :

```

PORT=3000

JWT_SECRET=votre_clé_secrète

```

### 4. Créer les dossiers nécessaires :

```

mkdir -p public/uploads

mkdir -p data

```

### 5. Démarrer l'application :

```

npm start

```

ou en mode développement :

```

npm run dev

```

### 8.3. Déploiement en production

-----

Pour un déploiement en production, plusieurs options sont possibles :

#### 1. Hébergement sur un VPS (Digital Ocean, AWS EC2, etc.) :

- Installer Node.js sur le serveur
- Cloner le dépôt et installer les dépendances

- Utiliser PM2 pour gérer le processus Node.js :

...

```
npm install -g pm2
```

```
pm2 start app.js --name "recipe-app"
```

...

- Configurer un reverse proxy (Nginx, Apache) pour exposer l'application

## 2. Hébergement sur une plateforme PaaS (Heroku, Vercel, etc.) :

- Suivre les instructions spécifiques à la plateforme
- Configurer les variables d'environnement
- S'assurer que les fichiers de données sont persistants

## 8.4. Considérations pour la production

-----

### 1. Sécurité :

- Utiliser HTTPS
- Configurer des en-têtes de sécurité (helmet)
- Utiliser une clé JWT forte et la garder secrète
- Limiter les tentatives de connexion (rate limiting)

### 2. Performance :

- Envisager une migration vers une base de données réelle
- Mettre en place un système de cache
- Optimiser les images téléchargées

### 3. Sauvegarde :

- Mettre en place des sauvegardes régulières des fichiers de données
- Sauvegarder les images téléchargées

=====

## 9. BONNES PRATIQUES ET OPTIMISATIONS

---

### 9.1. Bonnes pratiques de code

---

#### 1. Structure du projet :

- Organisation claire en dossiers fonctionnels (MVC)
- Séparation des préoccupations (routes, contrôleurs, modèles)
- Modularité et réutilisation du code

#### 2. Gestion des erreurs :

- Try/catch pour les opérations asynchrones
- Messages d'erreur explicites
- Logging des erreurs pour le débogage

#### 3. Asynchronicité :

- Utilisation de async/await pour la clarté du code
- Gestion correcte des promesses
- Éviter le callback hell

#### 4. Validation des données :

- Validation des entrées utilisateur
- Conversion des types de données
- Sanitization pour éviter les injections

### 9.2. Optimisations possibles

---

#### 1. Performance :

- Mise en cache des résultats fréquemment demandés

- Pagination pour limiter la taille des réponses
- Optimisation des requêtes de données

## 2. Base de données :

- Migration vers une base de données comme MongoDB ou PostgreSQL
- Indexation pour des recherches plus rapides
- Relations entre les entités

## 3. Images :

- Compression des images téléchargées
- Génération de plusieurs tailles (thumbnails)
- Utilisation d'un CDN pour servir les images

## 4. Frontend :

- Développement d'une interface utilisateur React/Vue/Angular
- Optimisation pour mobile (responsive design)
- Amélioration de l'expérience utilisateur

## 9.3. Tests et qualité de code

-----

### 1. Tests unitaires :

- Tests des modèles et fonctions utilitaires
- Utilisation de frameworks comme Jest ou Mocha

### 2. Tests d'intégration :

- Tests des API endpoints
- Simulation des requêtes HTTP

### 3. Documentation :

- Documentation des API (Swagger/OpenAPI)

- Commentaires de code JSDoc
- README et guides d'utilisation

=====

## 10. CONCLUSION

-----

Ce projet de site de recettes de cuisine est une application Node.js/Express complète qui démontre l'utilisation de nombreuses technologies et pratiques modernes de développement web. Bien qu'il utilise des fichiers JSON pour le stockage des données (ce qui le rend facile à déployer et à tester), il pourrait facilement être adapté pour utiliser une base de données traditionnelle pour des applications à plus grande échelle.

L'architecture MVC (Modèle-Vue-Contrôleur) utilisée dans ce projet facilite la maintenance et l'évolution du code. La séparation claire des responsabilités entre les différentes couches de l'application permet d'ajouter de nouvelles fonctionnalités ou de modifier les existantes sans impacter l'ensemble du système.

Le système d'authentification basé sur JWT offre une solution sécurisée et sans état pour gérer les sessions utilisateur, ce qui est idéal pour les API RESTful. Les fonctionnalités de gestion des recettes et des favoris offrent une expérience utilisateur complète et intuitive.

Pour aller plus loin, ce projet pourrait être étendu avec des fonctionnalités comme le partage social, les catégories de recettes, les notes et évaluations, ou encore une interface d'administration pour la modération du contenu.

=====

## ANNEXE : EXEMPLES DE CODE JAVASCRIPT

=====

Voici quelques exemples de code JavaScript qui illustrent des concepts importants utilisés dans le projet :

### 1. Middleware d'authentification (reconstitué) :

```
` ``javascript
```

```
const jwt = require('jsonwebtoken');
const UserModel = require('../models/userModel');

module.exports = async (req, res, next) => {
 try {
 // Extraction du token

 const authHeader = req.headers.authorization;
 if (!authHeader || !authHeader.startsWith('Bearer ')) {
 return res.status(401).json({
 success: false,
 message: 'Accès non autorisé, token manquant'
 });
 }
 }

 const token = authHeader.split(' ')[1];

 // Vérification du token
 const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret_key');

 // Récupération de l'utilisateur
 const user = await UserModel.getUserById(decoded.id);
 if (!user) {
 return res.status(401).json({
 success: false,
 message: 'Utilisateur non trouvé'
 });
 }

 // Ajout de l'utilisateur à la requête
 req.user = user;
 next();
}
```

```

} catch (error) {
 console.error('Erreur d\'authentification:', error);
 res.status(401).json({
 success: false,
 message: 'Token invalide ou expiré',
 error: error.message
 });
}
};
` ``

```

## 2. Fonction de génération de token JWT (reconstitué) :

```

` `` javascript
const jwt = require('jsonwebtoken');

function generateToken(userId) {
 // Création du payload
 const payload = {
 id: userId,
 iat: Date.now() / 1000, // Issued at (timestamp en secondes)
 exp: (Date.now() / 1000) + (60 * 60 * 24) // Expiration (24 heures)
 };

 // Génération du token
 const token = jwt.sign(payload, process.env.JWT_SECRET || 'secret_key');

 return token;
}

module.exports = generateToken;
` ``

```

### 3. Fonction de hachage de mot de passe avec bcrypt :

```
```javascript
const bcrypt = require('bcryptjs');

async function hashPassword(password) {
  // Génération d'un sel aléatoire
  const salt = await bcrypt.genSalt(10);

  // Hachage du mot de passe avec le sel
  const hashedPassword = await bcrypt.hash(password, salt);

  return hashedPassword;
}

async function comparePasswords(plainPassword, hashedPassword) {
  // Comparaison du mot de passe en clair avec le hash
  const isMatch = await bcrypt.compare(plainPassword, hashedPassword);

  return isMatch;
}

module.exports = {
  hashPassword,
  comparePasswords
};
```
```

### 4. Fonction de lecture/écriture de fichier JSON :

```
```javascript
const fs = require('fs');
```

```
const path = require('path');

const dataPath = path.join(__dirname, 'data.json');

async function readJsonFile() {
  try {
    // Vérification si le fichier existe
    if (!fs.existsSync(dataPath)) {
      // Création du fichier avec un tableau vide
      await fs.promises.writeFile(dataPath, JSON.stringify([], null, 2));
      return [];
    }

    // Lecture du fichier
    const data = await fs.promises.readFile(dataPath, 'utf8');

    // Parsing du JSON
    return JSON.parse(data);
  } catch (error) {
    console.error('Erreur lors de la lecture du fichier JSON:', error);
    throw error;
  }
}

async function writeJsonFile(data) {
  try {
    // Conversion des données en JSON formaté
    const jsonData = JSON.stringify(data, null, 2);

    // Écriture dans le fichier
    await fs.promises.writeFile(dataPath, jsonData);
  }
}
```

```

    return true;
  } catch (error) {
    console.error('Erreur lors de l\'écriture du fichier JSON:', error);
    throw error;
  }
}

```

```

module.exports = {
  readJsonFile,
  writeJsonFile
};
...

```

5. Configuration de multer pour le téléchargement d'images :

```

```javascript
const multer = require('multer');
const path = require('path');

// Configuration du stockage
const storage = multer.diskStorage({
 // Dossier de destination
 destination: (req, file, cb) => {
 cb(null, path.join(__dirname, 'public', 'uploads'));
 },
 // Nom du fichier
 filename: (req, file, cb) => {
 // Utilisation d'un timestamp pour éviter les conflits de noms
 const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
 // Récupération de l'extension du fichier original
 const ext = path.extname(file.originalname);

```

```

 // Création du nom de fichier
 cb(null, uniqueSuffix + ext);
 }
});

// Filtre pour n'accepter que les images
const fileFilter = (req, file, cb) => {
 // Vérification du type MIME
 if (file.mimetype.startsWith('image/')) {
 cb(null, true);
 } else {
 cb(new Error('Seules les images sont autorisées'), false);
 }
};

// Configuration de multer
const upload = multer({
 storage,
 fileFilter,
 limits: {
 fileSize: 5 * 1024 * 1024 // Limite à 5 Mo
 }
});

module.exports = upload;
...

```

Ces exemples illustrent les techniques et bonnes pratiques JavaScript utilisées dans le projet, notamment la gestion asynchrone avec `async/await`, la manipulation de fichiers, l'authentification sécurisée et le traitement des téléchargements de fichiers.